เอกสารข้อกำหนดโครงการและแผนการดำเนินงาน (System Requirements Specification & Roadmap) **ชื่อโครงการ**: FxGaze (AI-Powered Guitar Effect Analyzer) **เวอร์ชัน**: 1.0.0 **วันที่ปรับปรุงล่าสุด**: 14 **ตุลาคม** 2026

1. **ภาพรวมโครงการ** (Project Overview) FxGaze **คือเว็บแอปพลิเคชันสำหรับนักดนตรีและวิศวกรเสียง ที่ใช้เทคโนโลยีปัญญาประดิษฐ์** (AI) **และการประมวลผลสัญญาณเสียงดิจิทัล** (DSP) **ในการวิเคราะห์ไฟล์เสียงหรือสัญญาณสดจากกีตาร์ ระบบจะทำการแยกแยะประเภทของเอฟเฟกต์กีตาร์** (เช่น Reverb, Delay, Fuzz) **สกัดค่าพารามิเตอร์ทางฟิสิกส์ และแสดงผลลัพธ์ผ่านหน้าจอผู้ใช้งานในรูปแบบของก้อนเอฟเฟกต์จำลอง** (Interactive Stompbox) **ที่สามารถปรับค่าและตอบสนองได้เสมือนจริง**

2. **สถาปัตยกรรมระบบและเทคโนโลยี** (Tech Stack) 2.1 Frontend (**ส่วนหน้าบ้าน**) Framework: Next.js (React)

UI/Styling: Tailwind CSS **สำหรับออกแบบโครงสร้าง** Stompbox **และหน้าต่างวิเคราะห์**

Audio Visualization: Wavesurfer.js **สำหรับแสดงกราฟคลื่นเสียง** (Waveform) **และการเลือกช่วงเวลา** (Crop)

Audio Capture: Web Audio API (MediaRecorder) **สำหรับรับสัญญาณสดจาก** Audio Interface

2.2 Backend (**ส่วนหลังบ้าน**) Framework: FastAPI (Python) **เพื่อรองรับการทำงานแบบ** Asynchronous **ที่รวดเร็ว**

Task Queue: Celery **ร่วมกับ** Redis **สำหรับจัดคิวการวิเคราะห์เสียงเพื่อไม่ให้เกิด** Timeout

Audio Processing Engine:

Librosa: **สำหรับสกัดฟีเจอร์ทางฟิสิกส์** (ZCR, RMS Energy, Spectral Centroid, RT60)

HTDemucs: **สำหรับแยกแทร็กกีตาร์ออกจากไฟล์เพลงเต็ม** (Stem Separation)

AI Engine: Pathumma 8B (**รันแบบ** Local **ผ่าน** transformers **หรือ** Ollama) **สำหรับประมวลผลข้อมูลตัวเลขให้กลายเป็นบริบททางดนตรีและคำแนะนำ**

3. **ข้อกำหนดระบบ** (System Requirements) 3.1 **ข้อกำหนดด้านฟังก์ชันการทำงาน** (Functional Requirements - FR) FR-01 (Audio Input): **ระบบต้องรองรับการรับข้อมูลเสียง 2 รูปแบบ คือ การอัปโหลดไฟล์** (.wav, .mp3, .webm) **และการบันทึกเสียงสดผ่านเบราว์เซอร์**

FR-02 (Waveform Interaction): **ระบบต้องแสดงคลื่นเสียงบนเว็บได้ทันทีที่ได้รับข้อมูล และผู้ใช้ต้องสามารถลากเมาส์เพื่อเลือกเฉพาะช่วงเวลาที่ต้องการวิเคราะห์ได้**

FR-03 (Stem Separation): **หากผู้ใช้อัปโหลดเพลงเต็ม ระบบต้องสามารถแยกเฉพาะเสียงกีตาร์ออกมาเพื่อวิเคราะห์ได้**

FR-04 (Feature Extraction): Backend **ต้องสามารถคำนวณและสกัดค่าทางฟิสิกส์ของเสียงได้อย่างน้อย** 3 **ค่าหลัก ได้แก่ ความกังวาน** (Decay/RT60), **อัตราความแตกพร่า** (Zero-Crossing Rate), **และความถี่ศูนย์กลาง** (Spectral Centroid)

FR-05 (AI Inference): **ระบบต้องแปลงค่าที่สกัดได้เป็น** Text Prompt **เพื่อส่งให้โมเดล** Pathumma 8B **วิเคราะห์ และรับผลลัพธ์กลับมาเป็นรูปแบบ** JSON **เสมอ**

FR-06 (Dynamic UI): Frontend **ต้องอ่านค่า** JSON **ที่ได้จาก** AI **แล้วสร้าง** UI **ก้อนเอฟเฟกต์** (Stompbox) **ตามประเภทที่ตรวจพบ พร้อมหมุนค่า** Knobs **ให้ตรงกับพารามิเตอร์โดยอัตโนมัติ**

3.2 **ข้อกำหนดด้านเทคนิค** (Non-Functional Requirements - NFR) NFR-01 (Performance): **การวิเคราะห์เสียง 1 ท่อน** (ความยาวไม่เกิน 15 วินาที) **ต้องใช้เวลาประมวลผลรวมทั้งหมดไม่เกิน** 30 **วินาที เมื่อรันบนฮาร์ดแวร์** Local **ที่มี** GPU

NFR-02 (Asynchronous Flow): **ฝั่งผู้ใช้งานต้องไม่พบปัญหาหน้าเว็บค้างระหว่างรอ** AI **ประมวลผล โดยระบบต้องมีสถานะ** (Loading/Queued) **แจ้งให้ทราบ**

NFR-03 (Security & Privacy): การประมวลผลไฟล์เสียงและโมเดล AI ต้องเกิดขึ้นบนเครื่อง Local Server ทั้งหมด โดยจะไม่มีการส่งไฟล์เสียงของผู้ใช้ออกไปยัง Cloud ภายนอก

4. แผนการดำเนินงาน (Step-by-Step Implementation Roadmap) Phase 1: Foundation & Backend Setup (การวางโครงสร้างหลังบ้าน) ติดตั้งโปรเจกต์ Python สร้าง Virtual Environment และติดตั้ง Dependency หลัก (fastapi, uvicorn, redis, celery)

สร้างไฟล์ main.py และออกแบบ API Endpoint พื้นฐาน /api/analyze-tone สำหรับรับไฟล์ UploadFile

ติดตั้งและสตาร์ต Redis Server บนเครื่อง Local

เขียนโค้ดตั้งค่า Celery Worker เพื่อแยกการทำงานของการประมวลผลไฟล์เสียงออกจาก Main Thread ของ API

Phase 2: Audio Engineering Module (ระบบสกัดสัญญาณเสียง) ติดตั้ง librosa และ soundfile สำหรับอ่านไฟล์เสียง

สร้างคลาส AudioFeatureExtractor ที่มีฟังก์ชันสกัดค่า ZCR (จับเสียง Fuzz/Distortion) และ RMS/Decay (จับเสียง Reverb/Delay)

(Optional) ติดตั้ง demucs และเขียนฟังก์ชันย่อยสำหรับแยกเสียงกีตาร์ กรณีผู้ใช้อัปโหลดไฟล์ที่มีเครื่องดนตรีผสม

ทดสอบรัน Module นี้แยกต่างหากเพื่อดูความแม่นยำของตัวเลขที่สกัดได้

Phase 3: AI Engine Integration (ระบบปัญญาประดิษฐ์) โหลดโมเดล Pathumma 8B ลงในเครื่องผ่าน Ollama (แนะนำรุ่น pathumma-thaillm-8b-think Q4_K_M) หรือใช้ไลบรารี transformers

พัฒนาคลาส PathummaAnalyzer เพื่อรับข้อมูลจาก Phase 2

เขียน System Prompt ที่ผูกขาดการทำงานให้ออกมาเป็น JSON (กำหนด Schema: effect_type, parameters, description, advice)

เชื่อมต่อ Phase 2 และ Phase 3 เข้าด้วยกันภายใน Celery Task ให้ทำงานต่อเนื่องเป็น Pipeline เดียว

Phase 4: Frontend Development (การสร้างหน้าเว็บแอปพลิเคชัน) เริ่มต้นโปรเจกต์ Next.js (npx create-next-app) พร้อมติดตั้ง Tailwind CSS

ติดตั้งและนำเข้า wavesurfer.js เพื่อสร้าง Component WaveformPlayer ที่รองรับการอัปโหลดไฟล์และแสดงกราฟเสียง

สร้างปุ่ม "อัดเสียงสด" โดยใช้ navigator.mediaDevices.getUserMedia เพื่อดึงสัญญาณจาก Audio Interface

เขียนฟังก์ชันแปลงเสียงสด (Blob) เป็นไฟล์ FormData และสร้างฟังก์ชัน Fetch ยิงไปที่ Endpoint ของ Backend

Phase 5: Interactive UI & Data Binding (การแสดงผล Stompbox) ออกแบบ React Component ชื่อ PedalBox.jsx โดยให้รองรับการเปลี่ยนสีตาม Props effect_type

สร้าง Component ย่อย Knob.jsx ที่สามารถรับค่า (Value) 0-100 หรือหน่วยมิลลิวินาที มาหมุนภาพกราฟิกให้ตรงจุด

เขียน State Management ดักจับ Response JSON จาก Backend และส่งข้อมูล (Data Binding) ลงไปยัง PedalBox และ Knob

แสดงคำแนะนำ (advice) จาก AI ในกล่องข้อความด้านล่างก้อนเอฟเฟกต์

Phase 6: Optimization & Polish (การปรับแต่งและทดสอบ) ทดสอบระบบแบบ End-to-End โดยเสียบกีตาร์เข้า Audio Interface อัดเสียงสด และดูการเปลี่ยนแปลงของ UI

ปรับจูน Prompt ของ Pathumma อีกครั้งหากพบว่า AI ตีความพารามิเตอร์ผิดพลาด

จัดระเบียบ Error Handling เช่น กรณีผู้ใช้อัปโหลดไฟล์เสียงเงียบ หรือไฟล์ที่ไม่ใช่เสียงดนตรี ให้ระบบแจ้งเตือนอย่างถูกต้อง

ทำ Code Refactoring และบันทึกคู่มือการใช้งาน (Documentation) เป็นอันเสร็จสิ้นโครงการ